Saveetha Institute of Medical and Technical Sciences (SIMATS Engineering)
Department of Computer Science and Engineering

Item	Details
Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform ("ShopSphere")
Mode / Duration	Individual / 3 Days
Student Name	Chandra Prakash D
Student Reg. No.	192524060

Assumptions and Notation

- log means $\log_2$ throughout. Sizes: n = 100,000 (today) and n = 1,000,000 (next year).
- The speed in the question paper reads "10 basic operations per second"; the exponent was lost in the PDF. I assume **10^8 basic operations per second** (a typical figure for this type of exercise). Part (g) also shows what changes at 10^9 ops/s.
- The nightly processing window is 10 minutes = 600 s, i.e. at most 6×10^{10} basic operations at 10^8 ops/s.
- Base cases: T(1) = 1 for M3-S2 and M4; T(1) = 0 for the M5 recursion tree (this only affects the lower-order term).

(a) Understanding the Problem

For each module, the table gives the input size, the dominant basic operation, where it occurs, and the expected growth of its execution count.

Module	Input size	Dominant basic operation	Where it occurs	Expected growth
M1 Duplicate detector	n = number of products	Comparison of title and seller of Product[i] and Product[j]	Innermost line of the nested FOR loops; executed once for every pair (i, j) with i < j	Quadratic: about $n^2/2$. Doubling n gives about 4× the work
M2 API rate limiter	n = number of requests (window limit)	Audit(Request[j])	Inner FOR loop; it runs i times in the pass where i = 1, 2, 4, …, i < n	Linear: a geometric sum of about n to 2n. Doubling n gives about 2×
M3 Product search	n = number of products (or index entries)	Comparison of the search key with a catalogue/index entry	S1: each step of the sequential scan. S2: each halving step of the sorted-index search	S1: linear in n. S2: logarithmic, about $\log_2 n$
M4 Recommendation engine	n = size of the customer–product data	Merge/combine step costing n per call, plus the recursive calls	Every recursive level: a = 4 sub-problems of size n/2 (current) or a = 2 (redesign), plus linear merge	Current: quadratic (n^2). Redesign: n log n
M5 Sales analytics	n = number of sales records	Comparison/move in sorting the records at each level (n log n per	Sort at every level of the halving recursion; report generator f(n)	Recursion: $n \log^2 n$. Report generator: quadratic

Module	Input size	Dominant basic operation	Where it occurs	Expected growth
		call); report generator: the dominant $3n^2$ term		

Justification. In every module, the basic operation is the statement inside the innermost loop (M1, M2) or the work done outside the recursive calls (M3–M5). That operation is executed the largest number of times, so its count determines the growth rate. Constant-time statements such as loop counters or assignments are executed fewer times and do not change the order of growth.

(b) Analysing the Iterative Modules

Module M1 – Duplicate-Listing Detector

Basic operation: the comparison inside the inner loop. For a fixed i, j runs from i + 1 to n − 1, which is (n − 1 − i) iterations.

$$C(n) = \Sigma_{i=0}^{n-2} (n - 1 - i)$$

Let k = n − 1 − i. When i = 0, k = n − 1; when i = n − 2, k = 1.

$$C(n) = \Sigma_{k=1}^{n-1} k = (n - 1)\cdot n / 2 = (n^2 - n) / 2$$

> **Exact comparisons:** $C(n) = n(n - 1)/2$. **Time complexity:** best = average = worst = $\Theta(n^2)$ (the loops do not depend on the data). **Space complexity:** $\Theta(1)$, only the counters i and j are used (no extra data structure).

Interpretation. For n = 1,000,000 this is about 5×10^{11} comparisons. Doubling the catalogue quadruples the nightly run, so M1 is a scalability risk.

Module M2 – API Rate Limiter

Basic operation: Audit(Request[j]). In the pass where the outer variable equals i, the inner loop executes exactly i times (j = 0 … i − 1).

The outer variable takes the values $i = 2^0, 2^1, 2^2, \ldots, 2^{m-1}$, as long as $i < n$. The number of passes is $m = \lceil \log_2 n \rceil$.

$$A(n) = \Sigma_{k=0}^{m-1} 2^k = 2^m - 1 = 2^{\lceil \log_2 n \rceil} - 1$$

Since $n \le 2^{\lceil \log_2 n \rceil} < 2n$, we get $n - 1 \le A(n) \le 2n - 2$.

> **Exact count:** $A(n) = 2^{\lceil \log_2 n \rceil} - 1$ (= n − 1 when n is a power of 2). **Tight bound:** $\Theta(n)$. **Space:** $\Theta(1)$.

Is the team member's claim Θ(n log n) correct? No. The claim assumes the inner loop runs up to n times in every one of the log n passes. In fact the inner loop runs i times, and i is small in the early passes (1, 2, 4, 8, …). Only the last pass is close to n. The costs form a geometric series whose sum is dominated by its largest term, about n, so the total is $\Theta(n)$. The claim gives a valid upper bound, $O(n \log n)$, but not a tight one. Formally, $A(n) \le 2n - 2 = O(n)$ and $A(n) \ge n - 1 = \Omega(n)$, so $A(n) = \Theta(n)$ and A(n) is not $\Omega(n \log n)$.

Verification for n = 1,024

Here i takes the values 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 (the value 1024 fails the test i < n). That is $m = \log_2 1024 = 10$ passes.

$$A(1024) = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = 1{,}023 = 2^{10} - 1$$

The formula $2^{\lceil \log_2 n \rceil} - 1 = 2^{10} - 1 = 1{,}023$ agrees. The claimed n log n would give $1024 \times 10 = 10{,}240$, which is about 10× too large. This confirms $\Theta(n)$.

(c) Recurrence by Substitution – Search Version S2

Recurrence: T(n) = T(n/2) + 1, with T(1) = 1. Assume n = 2^k for the exact solution.

Backward substitution

$T(n) = T(n/2) + 1$

$T(n/2) = T(n/4) + 1 \implies T(n) = T(n/4) + 1 + 1 = T(n/4) + 2$

$T(n/4) = T(n/8) + 1 \implies T(n) = T(n/8) + 3$

⋮

After k substitutions: $T(n) = T(n/2^k) + k$

The recursion stops when $n/2^k = 1$, i.e. $2^k = n$, so $k = \log_2 n$.

$T(n) = T(1) + \log_2 n = 1 + \log_2 n$

Check: $T(2) = T(1) + 1 = 2$ and $1 + \log_2 2 = 2$ ✓. $T(8) = T(4) + 1 = T(2) + 2 = T(1) + 3 = 4$ and $1 + \log_2 8 = 4$ ✓.

> **Exact closed form:** $T(n) = \log_2 n + 1$ (n a power of 2); for general n, $T(n) = \lfloor \log_2 n \rfloor + 1$. **Time complexity:** $\Theta(\log n)$.

Space complexity of the two implementations

Implementation	Time (worst case)	Extra space	Reason
Iterative (low, high, mid variables in a loop)	$\Theta(\log n)$	$\Theta(1)$	Only a fixed number of variables, no growing stack
Recursive	$\Theta(\log n)$	$\Theta(\log n)$	One stack frame per halving; the recursion depth is $\log_2 n + 1$ frames

Interpretation. At n = 1,000,000 the search needs at most $\lfloor \log_2 1{,}000{,}000 \rfloor + 1 = 20$ comparisons. The iterative version is preferable in production because it uses constant memory.

(d) Applying the Master Theorem

Master Theorem for $T(n) = aT(n/b) + f(n)$, with $a \geq 1$, $b > 1$:

- Case 1: $f(n) = O(n^{(\log_b a - \varepsilon)})$ for some $\varepsilon > 0 \implies T(n) = \Theta(n^{(\log_b a)})$.
- Case 2: $f(n) = \Theta(n^{(\log_b a)}) \implies T(n) = \Theta(n^{(\log_b a)} \cdot \log n)$.
- Case 3: $f(n) = \Omega(n^{(\log_b a + \varepsilon)})$ and $a \cdot f(n/b) \leq \delta \cdot f(n)$ for some $\delta < 1 \implies T(n) = \Theta(f(n))$.

M4 – Current design: T(n) = 4T(n/2) + n

a	b	f(n)	n^(log_b a)	Case
4	2	n	$n^{(\log_2 4)} = n^2$	Case 1

$f(n) = n = O(n^{(2 - \varepsilon)})$ with $\varepsilon = 1$, because $n = O(n^1)$. So f(n) grows polynomially slower than n^2, and case 1 applies.

> **$T(n) = \Theta(n^2)$.** Exact solution with T(1) = 1: $T(n) = 2n^2 - n$ (check: $T(2) = 4 \cdot 1 + 2 = 6 = 2 \cdot 4 - 2$ ✓).

M4 – Proposed redesign: T(n) = 2T(n/2) + n

a	b	f(n)	n^(log_b a)	Case
2	2	n	$n^{(\log_2 2)} = n$	Case 2

f(n) = n = $\Theta(n^1) = \Theta(n^{(\log_b a)})$, so case 2 applies.

> **T(n) = Θ(n log n).** Exact solution with T(1) = 1: $T(n) = n \log_2 n + n$.

What happens when the data doubles?

Design	Ratio T(2n)/T(n)	Meaning
Current, $\Theta(n^2)$	$(2n)^2/n^2 = 4$	Doubling the data makes the engine about 4× slower
Redesign, Θ(n log n)	$2n \cdot \log(2n) / (n \cdot \log n) = 2(1 + 1/\log n) \approx 2.1$ at $n = 10^6$	Doubling the data makes it only slightly more than 2× slower

At $n = 10^6$ the redesign needs about 2×10^7 operations instead of about 2×10^{12}, i.e. roughly 100,000 times fewer. The redesign also removes the replicated partitions, saving memory.

M5 – T(n) = 2T(n/2) + n log n

Here a = 2, b = 2, f(n) = n log n, and n^(log_b a) = n.

- Case 1: needs $f(n) = O(n^{(1-\varepsilon)})$. Not true, since n log n grows faster than n.
- Case 2 (basic form): needs f(n) = Θ(n). Not true, since n log n is not O(n).
- Case 3: needs $f(n) = \Omega(n^{(1+\varepsilon)})$. Not true, because the ratio f(n)/n = log n grows more slowly than any n^ε. The difference is only logarithmic, not polynomial.

So none of the three standard cases applies. f(n) lies in the gap between case 2 and case 3.

Extended Master Theorem (case 2 with logarithmic factor). If $f(n) = \Theta(n^{(\log_b a)} \cdot \log^k n)$ with $k \geq 0$, then $T(n) = \Theta(n^{(\log_b a)} \cdot \log^{(k+1)} n)$. Here $f(n) = n \cdot \log^1 n$, so k = 1:

$$T(n) = \Theta(n \cdot \log^{(1+1)} n) = \Theta(n \log^2 n)$$

Verification by recursion tree (n = 2^k)

At level i there are 2^i sub-problems, each of size n/2^i. Each sub-problem sorts its records at cost $(n/2^i) \cdot \log(n/2^i)$. The cost per level is:

$$2^i \cdot (n/2^i) \cdot \log(n/2^i) = n \cdot (\log n - i)$$

There are log n levels (i = 0, …, log n − 1) of non-leaf work, plus n leaves of constant cost.

$$T(n) = \Sigma_{i=0}^{\log n - 1} n(\log n - i) + n \cdot T(1)$$
$$= n \cdot (\log n + (\log n - 1) + \ldots + 1) = n \cdot \log n \cdot (\log n + 1) / 2$$
$$= (1/2)\, n \log^2 n + (1/2)\, n \log n \quad (+ n \cdot T(1))$$

> **$T(n) = \Theta(n \log^2 n)$**, consistent with the extended Master Theorem. At $n = 10^6$ it is about 2.1×10^8 operations, which is cheap compared with the quadratic modules.

(e) Best, Average and Worst Case Analysis

S1 – Sequential search (unsorted catalogue)

Case	Input condition	Comparisons
Best	The searched product is the first element	$1 = \Theta(1)$
Average (successful)	Product present, each of the n positions equally likely	$(n + 1)/2 = \Theta(n)$
Worst	Product is the last element, or not present at all	$n = \Theta(n)$

Average case for S1 with probability p of the product existing

If the product is present (probability p), it is equally likely to be at any position i = 1, …, n, each with probability p/n, and finding it at position i costs i comparisons. If it is absent (probability 1 − p), all n elements are compared.

$$A(n) = \Sigma_\{i=1\}^\{n\}\ i \cdot (p/n) + n \cdot (1 - p)$$
$$= (p/n) \cdot n(n + 1)/2 + n(1 - p)$$
$$= p(n + 1)/2 + n(1 - p)$$

Check: p = 1 gives (n + 1)/2 (always found); p = 0 gives n (always absent). ✓

Evaluation for n = 1,000,000 and p = 0.5:

$$A = 0.5 \times 1{,}000{,}001 / 2 + 1{,}000{,}000 \times 0.5 = 250{,}000.25 + 500{,}000 = 750{,}000.25$$

> **About 750,000 comparisons on average per search**, i.e. 0.0075 s at 10^8 ops/s. This is acceptable for a single search but heavy under thousands of concurrent searches.

S2 – Binary search (sorted index)

Case	Input condition	Comparisons
Best	The key is exactly the middle entry of the index	$1 = \Theta(1)$
Average	Key equally likely to be at any position (or unsuccessful search)	about $\log_2 n - 1$ (successful) to $\log_2 n$ (unsuccessful) $= \Theta(\log n)$
Worst	Key is found at the last halving step or is absent	$\lfloor \log_2 n \rfloor + 1 = \Theta(\log n)$

For n = 1,000,000: best = 1, average ≈ 19 comparisons, worst = ⌊19.93⌋ + 1 = 20. Compare this with S1's average of 750,000: S2 is about 40,000 times faster, provided the index is kept sorted.

(f) Proving an Asymptotic Bound

Report generator cost: $f(n) = 3n^2 + 10n \log n + 50$ (log = $\log_2$).

Definition. $f(n) = \Theta(n^2)$ if there exist constants $c_1, c_2 > 0$ and n_0 such that $c_1 n^2 \le f(n) \le c_2 n^2$ for all $n \ge n_0$.

Lower bound (Ω)

Since $10n \log n \ge 0$ and $50 > 0$ for all $n \ge 1$, we have $f(n) \ge 3n^2$. So $c_1 = 3$ works for all $n \ge 1$.

Upper bound (O)

For $n \ge 1$: $\log_2 n \le n$, so $10n \log n \le 10n^2$. Also $50 \le 50n^2$. Therefore:

$$f(n) = 3n^2 + 10n \log n + 50 \le 3n^2 + 10n^2 + 50n^2 = 63n^2 \quad \text{for all } n \ge 1$$

So $c_2 = 63$ works.

> **Constants:** $c_1 = 3$, $c_2 = 63$, $n_0 = 1$: $3n^2 \le f(n) \le 63n^2$ for all $n \ge 1$. Hence $f(n) = \Theta(n^2)$.

Numerical check at n = 2: $f(2) = 12 + 10 \cdot 2 \cdot 1 + 50 = 82$, and $3 \cdot 4 = 12 \le 82 \le 63 \cdot 4 = 252$ ✓.

Showing f(n) is not O(n)

Proof by contradiction. Suppose $f(n) = O(n)$. Then there exist $c > 0$ and n_1 such that $f(n) \le c \cdot n$ for all $n \ge n_1$. We also know $f(n) \ge 3n^2$ for all $n \ge 1$. Combining these for $n \ge \max(1, n_1)$:

$$3n^2 \le f(n) \le c \cdot n \implies 3n \le c \implies n \le c/3$$

But this must hold for every $n \ge n_1$, and it fails for any $n > \max(n_1, c/3)$, e.g. $n = \lceil c/3 \rceil + n_1 + 1$. This is a contradiction, so $f(n)$ is not $O(n)$.

Equivalently, lim_{n→∞} f(n)/n = lim (3n + 10 log n + 50/n) = ∞, which is not a finite constant.

Interpretation. The nightly report generator grows quadratically, so a 10× larger catalogue costs about 100× more. At $n = 10^6$ it needs about 3×10^{12} operations.

(g) Scalability Decision

Operation counts are taken from the exact formulas derived above, with $n = 1{,}000{,}000$ and $\log_2 n \approx 19.93$. Time = operations / 10^8 ops/s. The window limit is 600 s.

Module	Operations at $n = 10^6$	Time at 10^8 ops/s	Within 10 min?	Complexity
M1 (duplicate detector)	$n(n-1)/2 \approx 5.0 \times 10^{11}$	≈ 5,000 s ≈ 83 min	**No – fails**	$\Theta(n^2)$
M2 (rate limiter)	$2^{20} - 1 = 1{,}048{,}575 \approx 1.05 \times 10^6$	≈ 0.0105 s	Yes	$\Theta(n)$
S2 (binary search)	$\lfloor \log_2 n \rfloor + 1 = 20$ per search	≈ 0.2 µs	Yes	$\Theta(\log n)$
M4 current	$2n^2 - n \approx 2.0 \times 10^{12}$	≈ 20,000 s ≈ 5.6 h	**No – fails**	$\Theta(n^2)$
M4 redesign	$n \log n + n \approx 2.09 \times 10^7$	≈ 0.21 s	Yes	$\Theta(n \log n)$
M5 (analytics)	≈ ½ n log n (log n + 1) ≈ 2.09×10^8	≈ 2.1 s	Yes	$\Theta(n \log^2 n)$

Extra observation: the report generator $f(n) = 3n^2 + \ldots$ needs about 3×10^{12} operations ≈ 30,000 s ≈ 8.3 h, so it would also miss the window, although it is not listed in the question's module list.

Growth between today and next year (n: $10^5 \to 10^6$)

Module	Time at $n = 10^5$	Time at $n = 10^6$	Growth
M1	≈ 50 s	≈ 5,000 s	100×
M4 current	≈ 200 s	≈ 20,000 s	100×
M4 redesign	≈ 0.017 s	≈ 0.21 s	≈ 12×
M5	≈ 0.14 s	≈ 2.1 s	≈ 15×

M1 and M4 (current) fit comfortably in the window today, but both grow 100× when the catalogue grows 10×, which pushes them far beyond it.

Recommendation

Modules that must be redesigned: M1 and M4 (current design). M2, S2, the M4 redesign and M5 can stay.

- **M1 – Duplicate-Listing Detector ($\Theta(n^2) \rightarrow \Theta(n)$):** replace the pairwise comparison by a hash table keyed on (title, seller). Scan the products once; if the key is already present, call MarkDuplicate; otherwise insert it. This gives expected $\Theta(n)$ time at the cost of $\Theta(n)$ extra space, about 10^6 operations instead of 5×10^{11}. An alternative without hashing is to sort by (title, seller) in $\Theta(n \log n)$ and mark adjacent equal entries. Either option finishes in well under a second.
- **M4 – Recommendation Engine:** adopt the proposed redesign that removes replicated partitions. This changes $T(n) = 4T(n/2) + n$, which is $\Theta(n^2)$, into $T(n) = 2T(n/2) + n$, which is $\Theta(n \log n)$, cutting 2×10^{12} operations down to about 2×10^7.
- **S1 – sequential search:** keep S2 (binary search over a sorted index) for production. Average 750,000 comparisons per query for S1 versus about 20 for S2.
- **M5:** acceptable (about 2 s). It could be improved to $\Theta(n \log n)$ by sorting once instead of at every level, which is optional.
- **Report generator:** $\Theta(n^2)$ as shown in (f); it should also be reviewed (for example, use incremental or aggregated queries).

Sensitivity to the server speed

If the servers actually run at 10^9 ops/s, M1 would take about 500 s (8.3 min), just inside the 10-minute window, but with almost no margin for growth. M4 (current) would still take 2,000 s (33 min) and fail. At 10^7 ops/s, M1, M4 (current) and M5 would all fail. Either way, the recommendations above hold, because $\Theta(n^2)$ modules grow 100× per 10× growth in data.

> **Conclusion:** $\Theta(n^2)$ modules (M1, current M4, report generator) will not scale to $n = 10^6$. $\Theta(n)$, $\Theta(n \log n)$, $\Theta(n \log^2 n)$ and $\Theta(\log n)$ modules (M2, redesigned M4, M5, S2) all run in a few seconds or less.

Code of Conduct

I, Chandra Prakash D (Reg. No. 192524060), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Chandra Prakash D Date: 10/10/2026